08 — LD-CONG Model Training (Jester Pretrain → LD-CONG Fine-tune)

Pipeline: Phase 1 Jester pretraining (27 classes) → Phase 2 LD-CONG fine-tuning (10 classes)

Anti-forgetting strategy: EWC (Elastic Weight Consolidation) with `EWC_WEIGHT=2000.0` — production setting that achieves only −0.9 pp Jester drop after fine-tuning (vs −13.1 pp for LLRD, −3.4 pp for progressive unfreezing).

Dataset: `data/processed/ld_cong_w16_dataset.npz` — built by `05_dataset_preparation_ld-cong.ipynb` with `WINDOW_LENGTH=16` frames (median LD-CONG segment ≈ 15 frames).

Classes (10): `palm`, `fist`, `thumb_up`, `shift_right`, `downward`, `upward`, `left`, `right`, `pinch`, `click`

Workflow

Phase	Stage	What trains	Epochs
1	Jester pretrain	All layers	`jester_config.training.epochs`
2A	Head warm-up	Head only (backbone frozen)	`FREEZE_EPOCHS`
2B	Full fine-tune	All layers + EWC penalty	`ld_cong_config.training.epochs`

> **Note on window length**: Jester was pretrained at 32 frames; LD-CONG uses 16 frames. `EnhancedTwoStreamLSTM` is sequence-length-agnostic (BiLSTM + temporal pooling), so the pretrained backbone loads and runs cleanly on shorter sequences.

In [25]:
```
import os
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import torch

REPO_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cw
sys.path.insert(0, str(REPO_ROOT / "src"))
```

```
plt.rcParams.update(
    {
        "figure.dpi": 110,
        "figure.facecolor": "white",
        "axes.spines.top": False,
        "axes.spines.right": False,
    }
)
sns.set_palette("tab10")

print(f"Repository root : {REPO_ROOT}")
print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available  : {torch.cuda.is_available()}")

if torch.cuda.is_available():
    DEVICE = torch.device("cuda")
elif torch.backends.mps.is_available():
    DEVICE = torch.device("mps")
else:
    DEVICE = torch.device("cpu")
print(f"Device          : {DEVICE}")
```

```
Repository root : /home/divclab/.hslay13/Maestro
PyTorch version : 2.6.0+cu124
CUDA available  : True
Device          : cuda
```

1. Configuration

In [26]:

```
# ── Config paths ──────────────────────────────────────────────────────
JESTER_CONFIG_PATH  = REPO_ROOT / "configs/model/two_stream_attn_jester.yaml
LD_CONG_CONFIG_PATH = REPO_ROOT / "configs/model/two_stream_attn_ld_cong.yam

# ── Dataset paths ─────────────────────────────────────────────────────
JESTER_NPZ       = REPO_ROOT / "data/processed/jester_pretrain_dataset.npz"
LD_CONG_RAW_NPZ  = REPO_ROOT / "data/processed/ld_cong_w16_dataset.npz"
LD_CONG_NPZ      = REPO_ROOT / "data/processed/ld_cong_hybrid_dataset.npz"
IPN_NPZ          = REPO_ROOT / "data/processed/ipn_dataset_v5_10class_mirror
VOCAB_CONFIG     = REPO_ROOT / "configs/gestures/vocabulary_ld_cong_hybrid.y

# ── Pretrained Jester checkpoint ──────────────────────────────────────
#
# Options:
#   1. Set to a .pt file from a prior notebook 03 / 07 run (fastest).
#   2. Set to None → Phase 1 Jester pretraining will run from scratch.
#
# Auto-detect latest Jester checkpoint in models/checkpoints/
_JESTER_CKPT_CANDIDATES = sorted(
    (REPO_ROOT / "models/checkpoints").glob("jester*.pt"),
    key=lambda p: p.stat().st_mtime if p.exists() else 0,
    reverse=True,
)
PRETRAINED_CHECKPOINT = _JESTER_CKPT_CANDIDATES[0] if _JESTER_CKPT_CANDIDATE
#
```

```
# To override: uncomment and set manually.
# PRETRAINED_CHECKPOINT = REPO_ROOT / "models/checkpoints/jester_two_stream_

# ── Fine-tuning hyper-parameters (Stage A + B) ──────────────────────────
FREEZE_EPOCHS          = 10   # Stage A: head-only warm-up epochs
STAGE_B_WARMUP_EPOCHS  = 5    # Stage B: backbone LR linear warmup (0 → targ
BACKBONE_LR_MULTIPLIER = 0.1  # Backbone LR = head LR × multiplier

# ── Anti-forgetting (production: EWC-only) ──────────────────────────────
# From catastrophic-forgetting experiments (branch fix/model-catastrophic-fo
#   EWC=2000  →  −0.9 pp Jester drop  ✓  (production choice)
#   LLRD      →  −13.1 pp             ✗
#   ProgUnfreeze 3→5 epochs → −3.4 to −5.6 pp  ✗
EWC_WEIGHT             = 2000.0  # Elastic Weight Consolidation strength
FORGETTING_PENALTY_WEIGHT = 0.5   # Composite early-stopping Jester penalty
DISTILLATION_WEIGHT    = 0.0    # Feature distillation — disabled
REPLAY_CE_WEIGHT       = 0.0    # Replay classification CE — disabled
REPLAY_MAX_SAMPLES_PER_CLASS = 500  # Jester replay buffer (used only if > 0
GPM_COMPONENTS         = 0      # Gradient Projection Memory — disabled

# ── Output directories ──────────────────────────────────────────────────
CHECKPOINT_DIR = REPO_ROOT / "models/checkpoints"
EXPORTED_DIR   = REPO_ROOT / "models/exported"
RUN_ROOT_DIR   = REPO_ROOT / "experiments/runs"

# ── Pre-flight validation ───────────────────────────────────────────────
print("=" * 70)
print("PRE-FLIGHT VALIDATION")
print("=" * 70)

validation_ok = True
required = [
    (JESTER_CONFIG_PATH,  "Jester config       "),
    (LD_CONG_CONFIG_PATH, "LD-CONG config      "),
    (JESTER_NPZ,          "Jester NPZ          "),
    (LD_CONG_RAW_NPZ,     "LD-CONG raw dataset "),
    (IPN_NPZ,             "IPN NPZ             "),
    (VOCAB_CONFIG,        "Hybrid vocab YAML   "),
]
for path, label in required:
    ok = path.exists()
    icon = "✓" if ok else "✗ MISSING"
    print(f"  {icon:<12} {label}  {path.name}")
    if not ok:
        validation_ok = False

if LD_CONG_NPZ.exists():
    print(f"  ✓            LD-CONG hybrid NPZ  {LD_CONG_NPZ.name}  (pre-buil
else:
    print(f"  ℹ            LD-CONG hybrid NPZ  {LD_CONG_NPZ.name}  (will be

if PRETRAINED_CHECKPOINT is not None:
    ok = PRETRAINED_CHECKPOINT.exists()
    icon = "✓" if ok else "✗ MISSING"
    print(f"  {icon:<12} Pretrained ckpt      {PRETRAINED_CHECKPOINT.name}")
    if not ok:
```

```
        print("  ⚠  Checkpoint not found — will run Phase 1 from scratch.")
        PRETRAINED_CHECKPOINT = None
else:
    print("  ℹ  No pretrained checkpoint — Phase 1 Jester pretraining will r

if not validation_ok:
    raise FileNotFoundError(
        "Missing required files.\n"
        "  • Run 05_dataset_preparation_ld-cong.ipynb to build ld_cong_w16_c
        "  • Run 04_dataset_preparation_jester_ipn.ipynb to build jester_pre
        "  • Run build_ipn_dataset.py to build ipn_dataset_v5_10class_mirror
        "  • Ensure configs/gestures/vocabulary_ld_cong_hybrid.yaml exists"
    )

print()
print(f"Device                   : {DEVICE}")
print(f"Freeze epochs            : {FREEZE_EPOCHS} (Stage A head-only warm-u
print(f"Stage B backbone warmup  : {STAGE_B_WARMUP_EPOCHS} epochs (0 → targe
print(f"Backbone LR multiplier   : {BACKBONE_LR_MULTIPLIER}×")
print(f"Pretrained checkpoint    : {PRETRAINED_CHECKPOINT}")
print()
print("Phase 2 continual-learning defences (Stage B):")
print(f"  EWC weight             : {EWC_WEIGHT}  (enabled)")
print(f"  Forgetting penalty     : {FORGETTING_PENALTY_WEIGHT}  (composite e
print(f"  Distillation weight    : {DISTILLATION_WEIGHT}  (disabled)")
print(f"  Replay CE weight       : {REPLAY_CE_WEIGHT}  (disabled)")
print(f"  GPM components         : {GPM_COMPONENTS}  (disabled)")
```

```
======================================================================
PRE-FLIGHT VALIDATION
======================================================================
  ✓            Jester config         two_stream_attn_jester.yaml
  ✓            LD-CONG config        two_stream_attn_ld_cong.yaml
  ✓            Jester NPZ            jester_pretrain_dataset.npz
  ✓            LD-CONG raw dataset   ld_cong_w16_dataset.npz
  ✓            IPN NPZ               ipn_dataset_v5_10class_mirrored.npz
  ✓            Hybrid vocab YAML     vocabulary_ld_cong_hybrid.yaml
  ℹ           LD-CONG hybrid NPZ  ld_cong_hybrid_dataset.npz  (will be built
below)
  ℹ  No pretrained checkpoint — Phase 1 Jester pretraining will run.

Device                   : cuda
Freeze epochs            : 10 (Stage A head-only warm-up)
Stage B backbone warmup  : 5 epochs (0 → target backbone LR)
Backbone LR multiplier   : 0.1×
Pretrained checkpoint    : None

Phase 2 continual-learning defences (Stage B):
  EWC weight             : 2000.0  (enabled)
  Forgetting penalty     : 0.5  (composite early stopping)
  Distillation weight    : 0.0  (disabled)
  Replay CE weight       : 0.0  (disabled)
  GPM components         : 0  (disabled)
```

In [27]:

```
# ── Section 2b helpers — load / remap / crop / extract / merge ──────────
import json
```

```python
from collections import Counter
import numpy as np

MAX_SAMPLES_PER_CLASS = 3000
BUILD_SEED            = 42
_rng = np.random.default_rng(BUILD_SEED)

def _load_npz(path) -> dict:
    print(f"  Loading {path.name} …", end=" ", flush=True)
    with np.load(path, allow_pickle=True) as f:
        d = {k: f[k] for k in f.files}
    for key in ("labels", "recording_ids", "sample_ids"):
        if key in d:
            d[key] = d[key].astype(str)
    print(f"{len(d.get('labels', []))} samples  shape={d['sequences'].shape}
    return d

def _remap_labels(data: dict, remap: dict) -> dict:
    """Rename labels using object dtype to avoid fixed-width unicode truncat
    labels = data["labels"].astype(object).copy()
    for old, new in remap.items():
        labels[labels == old] = new
    return {**data, "labels": labels}

def _center_crop(data: dict, target_len: int) -> dict:
    """Center-crop W=32 sequences to target_len frames."""
    w = data["sequences"].shape[1]
    if w == target_len:
        return data
    start = (w - target_len) // 2
    print(f"    Center-cropping W={w} → W={target_len}  (frames {start}:{sta
    return {**data, "sequences": data["sequences"][:, start:start + target_l

def _extract(data: dict, classes: list, prefix: str) -> dict:
    """Extract rows for the given classes, cap per-class, and prefix IDs."""
    labels = data["labels"]
    idxs   = np.where(np.isin(labels, classes))[0]
    if len(idxs) == 0:
        print(f"    ⚠ no samples for {classes} in '{prefix}'")
        return {}
    found = Counter(labels[idxs].tolist())
    print(f"    {prefix}: found  { {c: found.get(c, 0) for c in classes} }")
    kept: list = []
    for cls in classes:
        ci = idxs[labels[idxs] == cls]
        if MAX_SAMPLES_PER_CLASS > 0 and len(ci) > MAX_SAMPLES_PER_CLASS:
            ci = _rng.choice(ci, MAX_SAMPLES_PER_CLASS, replace=False)
            print(f"      capped {cls}: {found[cls]} → {MAX_SAMPLES_PER_CLAS
        kept.extend(ci.tolist())
    idxs = np.array(sorted(kept), dtype=np.int64)
    result = {k: v[idxs] for k, v in data.items()
              if k != "metadata_json" and hasattr(v, "__len__") and len(v) =
    result["recording_ids"] = np.array([f"{prefix}{r}" for r in result["recc
    result["sample_ids"]    = np.array([f"{prefix}{s}" for s in result["samp
    after = Counter(result["labels"].tolist())
    print(f"    {prefix}: kept   { dict(after) }  ({sum(after.values())} tot
```

```
    return result

def _merge(subsets: list, vocabulary: list, window_length: int, feature_dim:
    """Concatenate subsets and recompute label_ids from the hybrid vocabular
    label_to_id = {lbl: i for i, lbl in enumerate(vocabulary)}
    keys = ["sequences", "labels", "label_ids", "recording_ids", "sample_ids
            "start_timestamp_ms", "end_timestamp_ms", "valid_ratio"]
    bufs: dict = {k: [] for k in keys}
    for s in subsets:
        if not s:
            continue
        n     = len(s["labels"])
        zeros = np.zeros(n, dtype=np.int64)
        ones  = np.ones(n,  dtype=np.float32)
        bufs["sequences"].append(s["sequences"])
        bufs["labels"].append(s["labels"])
        bufs["label_ids"].append(
            np.array([label_to_id.get(l, -1) for l in s["labels"].tolist()],
        bufs["recording_ids"].append(s["recording_ids"])
        bufs["sample_ids"].append(s["sample_ids"])
        bufs["start_timestamp_ms"].append(s.get("start_timestamp_ms", zeros)
        bufs["end_timestamp_ms"].append(s.get("end_timestamp_ms",     zeros)
        bufs["valid_ratio"].append(s.get("valid_ratio",               ones))
    merged = {k: np.concatenate(v, axis=0) for k, v in bufs.items() if v}
    bad = (merged["label_ids"] == -1).sum()
    if bad:
        print(f"  ⚠ dropping {bad} samples with unknown labels")
        keep   = merged["label_ids"] != -1
        merged = {k: v[keep] for k, v in merged.items()}
    assert merged["sequences"].shape[1:] == (window_length, feature_dim), \
        f"Shape mismatch: {merged['sequences'].shape}"
    merged["metadata_json"] = np.asarray(json.dumps({
        "feature_schema_version": "feature-schema-v5",
        "feature_dim": feature_dim, "window_length": window_length,
        "label_strategy": "ld_cong_hybrid_merge",
        "vocabulary": vocabulary, "seed": BUILD_SEED,
    }), dtype=object)
    return merged

print("✓ Helper functions defined.")
```

```
✓ Helper functions defined.
```

In [28]:

```
# ── Section 2c — Build hybrid dataset (LD-CONG + Jester + IPN) ──────────
import yaml

if LD_CONG_NPZ.exists():
    print(f"✓ Hybrid dataset already exists: {LD_CONG_NPZ.name}")
    print("  Delete it and re-run to rebuild from scratch.")
else:
    # ── Vocabulary config ──────────────────────────────────────────────
    with VOCAB_CONFIG.open() as _f:
        _vcfg = yaml.safe_load(_f)

    _vocabulary    = [str(g) for g in _vcfg["gestures"]]
    _sources       = {cls: str(src) for cls, src in _vcfg.get("sources", {})
```

```
    _ld_remap      = _vcfg.get("label_remap", {}).get("ld_cong", {})

    _ld_only   = [c for c in _vocabulary if _sources.get(c) == "ld_cong"]
    _j_only    = [c for c in _vocabulary if _sources.get(c) == "jester"]
    _ipn_only  = [c for c in _vocabulary if _sources.get(c) == "ipn"]
    _both      = [c for c in _vocabulary if _sources.get(c) == "both"]

    print(f"Vocabulary  ({len(_vocabulary)} classes):")
    print(f"  LD-CONG only : {_ld_only}")
    print(f"  Jester only  : {_j_only}")
    print(f"  IPN only     : {_ipn_only}")
    print(f"  Pooled both  : {_both}")
    print(f"  Label remap  : {_ld_remap}\n")

    # ── LD-CONG ──────────────────────────────────────────────────────────
    print("LD-CONG:")
    _ld_data = _load_npz(LD_CONG_RAW_NPZ)
    _W       = int(_ld_data["sequences"].shape[1])   # 16
    _F       = int(_ld_data["sequences"].shape[2])   # 147
    _ld_data = _remap_labels(_ld_data, _ld_remap)
    _ld_sub  = _extract(_ld_data, _ld_only + _both, "ld_cong_")
    del _ld_data

    # ── Jester ───────────────────────────────────────────────────────────
    print("\nJester:")
    _j_data = _load_npz(JESTER_NPZ)
    _j_data = _center_crop(_j_data, _W)
    _j_sub  = _extract(_j_data, _j_only + _both, "jester_")
    del _j_data

    # ── IPN ──────────────────────────────────────────────────────────────
    print("\nIPN:")
    _ipn_data = _load_npz(IPN_NPZ)
    _ipn_data = _center_crop(_ipn_data, _W)
    _ipn_sub  = _extract(_ipn_data, _ipn_only, "ipn_")
    del _ipn_data

    # ── Merge ────────────────────────────────────────────────────────────
    print("\nMerging …")
    _merged  = _merge([_ld_sub, _j_sub, _ipn_sub], _vocabulary, _W, _F)
    _dist    = Counter(_merged["labels"].tolist())
    _n_tot   = sum(_dist.values())
    print(f"\nMerged: {_n_tot:,} samples  {len(_dist)} classes")
    for cls in _vocabulary:
        bar = "" * min(40, _dist.get(cls, 0) * 40 // max(1, max(_dist.value
        print(f"  {cls:<30} {_dist.get(cls, 0):5d}  {bar}")

    # ── Save ─────────────────────────────────────────────────────────────
    LD_CONG_NPZ.parent.mkdir(parents=True, exist_ok=True)
    np.savez_compressed(LD_CONG_NPZ, **_merged)
    _size_mb = LD_CONG_NPZ.stat().st_size / 1024**2
    print(f"\n✓ Saved → {LD_CONG_NPZ.name}  ({_size_mb:.1f} MB)")
```

```
Vocabulary  (14 classes):
  LD-CONG only : ['palm', 'fist', 'thumb_up', 'pinch', 'click']
  Jester only  : ['zooming_in_full_hand', 'zooming_out_full_hand']
  IPN only     : ['point_one', 'point_two', 'unknown']
  Pooled both  : ['swiping_right', 'swiping_left', 'swiping_down', 'swiping_
up']
  Label remap  : {'right': 'swiping_right', 'left': 'swiping_left', 'downwar
d': 'swiping_down', 'upward': 'swiping_up'}

LD-CONG:
  Loading ld_cong_w16_dataset.npz … 59316 samples  shape=(59316, 16, 147)
    ld_cong_: found  {'palm': 2218, 'fist': 1882, 'thumb_up': 12816, 'pinc
h': 1579, 'click': 1664, 'swiping_right': 5044, 'swiping_left': 6563, 'swipi
ng_down': 6383, 'swiping_up': 7526}
      capped thumb_up: 12816 → 3000
      capped swiping_right: 5044 → 3000
      capped swiping_left: 6563 → 3000
      capped swiping_down: 6383 → 3000
      capped swiping_up: 7526 → 3000
    ld_cong_: kept   {'palm': 2218, 'fist': 1882, 'thumb_up': 3000, 'swiping
_down': 3000, 'swiping_up': 3000, 'swiping_left': 3000, 'swiping_right': 300
0, 'pinch': 1579, 'click': 1664}  (22343 total)

Jester:
  Loading jester_pretrain_dataset.npz … 59549 samples  shape=(59549, 32, 14
7)
    Center-cropping W=32 → W=16  (frames 8:24)
    jester_: found  {'zooming_in_full_hand': 4349, 'zooming_out_full_hand':
4499, 'swiping_right': 629, 'swiping_left': 760, 'swiping_down': 850, 'swipi
ng_up': 1024}
      capped zooming_in_full_hand: 4349 → 3000
      capped zooming_out_full_hand: 4499 → 3000
    jester_: kept   {'zooming_in_full_hand': 3000, 'zooming_out_full_hand':
3000, 'swiping_left': 760, 'swiping_right': 629, 'swiping_up': 1024, 'swipin
g_down': 850}  (9263 total)

IPN:
  Loading ipn_dataset_v5_10class_mirrored.npz … 124488 samples  shape=(12448
8, 32, 147)
    Center-cropping W=32 → W=16  (frames 8:24)
    ipn_: found  {'point_one': 54002, 'point_two': 51759, 'unknown': 3198}
      capped point_one: 54002 → 3000
      capped point_two: 51759 → 3000
      capped unknown: 3198 → 3000
    ipn_: kept   {'point_one': 3000, 'point_two': 3000, 'unknown': 3000}  (9
000 total)

Merging …

Merged: 40,606 samples  14 classes
  palm                          2218  ████████████████████
  fist                          1882  ██████████████████
  thumb_up                      3000  ████████████████████████████
  pinch                         1579  ███████████████
  click                         1664  ████████████████
  swiping_right                 3629  ██████████████████████████████████
```

```
  swiping_left                3760 ████████████████████████████████
█
  swiping_down                3850 ████████████████████████████████
██
  swiping_up                  4024 ████████████████████████████████
████
  zooming_in_full_hand        3000 ████████████████████████
  zooming_out_full_hand       3000 ████████████████████████
  point_one                   3000 ████████████████████████
  point_two                   3000 ████████████████████████
  unknown                     3000 ████████████████████████

✓ Saved → ld_cong_hybrid_dataset.npz  (178.1 MB)
```

In [29]:

```python
# ── Section 2d — Class distribution chart ──────────────────────────
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

with np.load(LD_CONG_NPZ, allow_pickle=True) as _d:
    _lbl_all = _d["labels"].astype(str)
    _rid_all = _d["recording_ids"].astype(str)
    _vocab   = json.loads(str(_d["metadata_json"].item()))["vocabulary"]
    _W_meta  = json.loads(str(_d["metadata_json"].item()))["window_length"]
    _F_meta  = json.loads(str(_d["metadata_json"].item()))["feature_dim"]

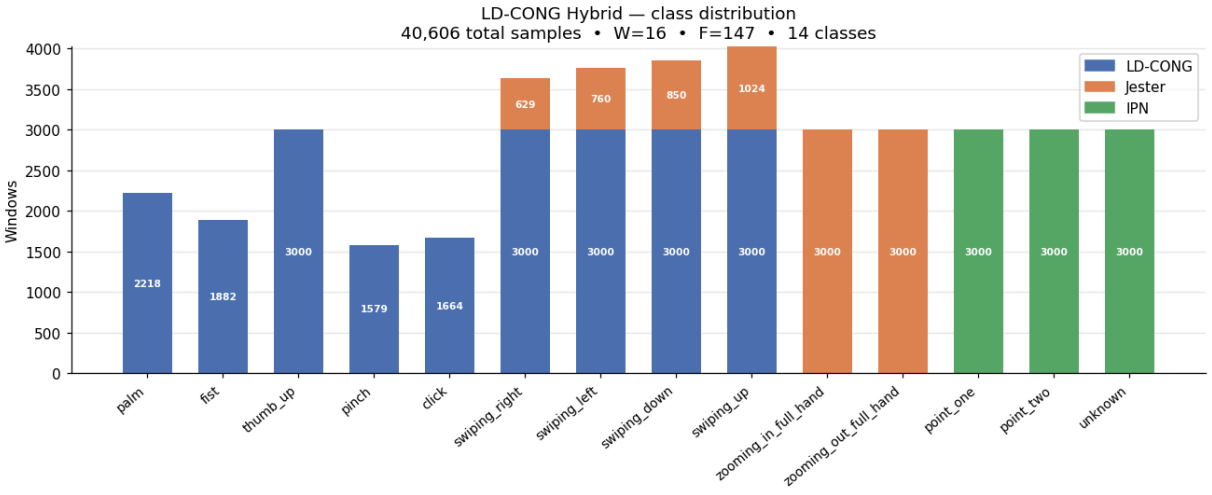
_src_cfg = {
    "ld_cong_": ("#4C72B0", "LD-CONG"),
    "jester_":  ("#DD8452", "Jester"),
    "ipn_":     ("#55A868", "IPN"),
}

_cnt = {src: [0] * len(_vocab) for src in _src_cfg}
for lbl, rid in zip(_lbl_all, _rid_all):
    if lbl not in _vocab:
        continue
    idx = _vocab.index(lbl)
    for src in _src_cfg:
        if rid.startswith(src):
            _cnt[src][idx] += 1
            break

fig, ax = plt.subplots(figsize=(12, 5))
x      = np.arange(len(_vocab))
bottom = np.zeros(len(_vocab))
for src, (colour, label) in _src_cfg.items():
    vals = np.array(_cnt[src])
    ax.bar(x, vals, bottom=bottom, color=colour, label=label, width=0.65)
    # annotate non-zero segments
    for i, (v, b) in enumerate(zip(vals, bottom)):
        if v > 0:
            ax.text(i, b + v / 2, str(v), ha="center", va="center",
                    fontsize=7, color="white", fontweight="bold")
    bottom += vals

ax.set_xticks(x)
ax.set_xticklabels(_vocab, rotation=40, ha="right", fontsize=9)
```

```
ax.set_ylabel("Windows")
ax.set_title(
    f"LD-CONG Hybrid — class distribution\n"
    f"{len(_lbl_all):,} total samples  •  W={_W_meta}  •  F={_F_meta}  •  {l
)
ax.legend(handles=[mpatches.Patch(color=c, label=l) for _, (c, l) in _src_cf
          loc="upper right")
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)
plt.tight_layout()
plt.show()
```



2. Load Configurations

In [30]:
```
from maestro.training.train_lstm import TrainingPipelineConfig, load_trainir

jester_config:  TrainingPipelineConfig = load_training_config(JESTER_CONFIG_
ld_cong_config: TrainingPipelineConfig = load_training_config(LD_CONG_CONFIG

print("=" * 70)
print("PHASE 1: JESTER PRETRAINING CONFIG (27 classes)")
print("=" * 70)
print(f"  Num classes      : {jester_config.model.num_classes}")
print(f"  Hidden size      : {jester_config.model.hidden_size}")
print(f"  Proj dim         : {jester_config.model.proj_dim}")
print(f"  Num heads        : {jester_config.model.num_heads}")
print(f"  Epochs           : {jester_config.training.epochs}")
print(f"  LR               : {jester_config.training.learning_rate:.2e}")
print(f"  Max/class        : {jester_config.training.max_samples_per_class}"

print()
print("=" * 70)
print("PHASE 2: LD-CONG FINE-TUNING CONFIG (10 classes)")
print("=" * 70)
print(f"  Num classes      : {ld_cong_config.model.num_classes}")
print(f"  Hidden size      : {ld_cong_config.model.hidden_size}")
print(f"  Proj dim         : {ld_cong_config.model.proj_dim}")
print(f"  Num heads        : {ld_cong_config.model.num_heads}")
print(f"  Epochs           : {ld_cong_config.training.epochs}")
```

```
print(f"  LR              : {ld_cong_config.training.learning_rate:.2e}")
print(f"  Max/class       : {ld_cong_config.training.max_samples_per_class}
print(f"  Early stop pat.  : {ld_cong_config.training.early_stopping_patienc

# Validate backbone compatibility (hidden_size, proj_dim, num_heads, num_lay
mismatch = []
for attr in ("hidden_size", "proj_dim", "num_heads", "num_layers"):
    j = getattr(jester_config.model, attr, None)
    h = getattr(ld_cong_config.model, attr, None)
    if j != h:
        mismatch.append(f"{attr}: jester={j}, ld_cong={h}")
if mismatch:
    raise ValueError(
        "Backbone architecture mismatch between Jester and LD-CONG configs.\
        "Cannot load pretrained weights.\n" + "\n".join(mismatch)
    )
print()
print("✓ Backbone architecture compatible (pretrained weights will load clea
```

```
======================================================================
PHASE 1: JESTER PRETRAINING CONFIG (27 classes)
======================================================================
  Num classes      : 27
  Hidden size      : 96
  Proj dim         : 96
  Num heads        : 8
  Epochs           : 80
  LR               : 1.00e-04
  Max/class        : 4000

======================================================================
PHASE 2: LD-CONG FINE-TUNING CONFIG (10 classes)
======================================================================
  Num classes      : 14
  Hidden size      : 96
  Proj dim         : 96
  Num heads        : 8
  Epochs           : 60
  LR               : 3.00e-05
  Max/class        : 3000
  Early stop pat.  : 12

✓ Backbone architecture compatible (pretrained weights will load cleanly).
```

3. Inspect LD-CONG Dataset

In [31]:
```
import json as _json

from maestro.training.datamodule import GestureDataModule

raw_ld = np.load(LD_CONG_NPZ, allow_pickle=True)
seq_ld     = raw_ld["sequences"]
labels_ld  = raw_ld["labels"].astype(str)
label_ids  = raw_ld["label_ids"].astype(np.int64)
rec_ids    = raw_ld.get("recording_ids", np.array(["?"] * len(labels_ld))).a
```

```
N, T, F = seq_ld.shape
print(f"LD-CONG shape     : {seq_ld.shape}  (samples × window × features)")
print(f"Window length     : {T} frames  (WINDOW_LENGTH=16)")
print(f"Feature dims      : {F}  (feature-schema-v5)")
print(f"NaN count         : {np.isnan(seq_ld).sum()}")

meta_ld = {}
if "metadata_json" in raw_ld:
    meta_ld = _json.loads(str(raw_ld["metadata_json"].item()))
vocab_ld = list(meta_ld.get("vocabulary", sorted(set(labels_ld.tolist()))))

print(f"\nVocabulary ({len(vocab_ld)} classes):")
for i, cls in enumerate(vocab_ld):
    print(f"  {i:2d}  {cls}")

dm_ld = GestureDataModule(
    dataset_path=LD_CONG_NPZ,
    batch_size=ld_cong_config.training.batch_size,
    split=ld_cong_config.split,
    seed=ld_cong_config.training.seed,
    num_workers=0,
    pin_memory=False,
    max_samples_per_class=ld_cong_config.training.max_samples_per_class,
    exclude_classes=ld_cong_config.training.exclude_classes,
)
dm_ld.setup_indices_only(label_ids=label_ids, recording_ids=rec_ids, vocab=v

print(
    f"\nSplit sizes  : train={len(dm_ld.train_indices):,}"
    f"  val={len(dm_ld.val_indices):,}"
    f"  test={len(dm_ld.test_indices):,}"
)
print(f"Num classes  : {dm_ld.num_classes}")
```

```
LD-CONG shape     : (40606, 16, 147)  (samples × window × features)
Window length     : 16 frames  (WINDOW_LENGTH=16)
Feature dims      : 147  (feature-schema-v5)
NaN count         : 0

Vocabulary (14 classes):
   0  palm
   1  fist
   2  thumb_up
   3  pinch
   4  click
   5  swiping_right
   6  swiping_left
   7  swiping_down
   8  swiping_up
   9  zooming_in_full_hand
  10  zooming_out_full_hand
  11  point_one
  12  point_two
  13  unknown

Split sizes  : train=27,053  val=5,242  test=5,048
Num classes  : 14
```

In [32]:

```python
from collections import Counter

class_labels_ld = dm_ld.class_labels
dist_all   = Counter(labels_ld.tolist())
dist_train = Counter(labels_ld[dm_ld.train_indices].tolist())
dist_val   = Counter(labels_ld[dm_ld.val_indices].tolist())
dist_test  = Counter(labels_ld[dm_ld.test_indices].tolist())

counts_all   = [dist_all.get(c, 0)   for c in class_labels_ld]
counts_train = [dist_train.get(c, 0) for c in class_labels_ld]
counts_val   = [dist_val.get(c, 0)   for c in class_labels_ld]
counts_test  = [dist_test.get(c, 0)  for c in class_labels_ld]

x = np.arange(len(class_labels_ld))
w = 0.26

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

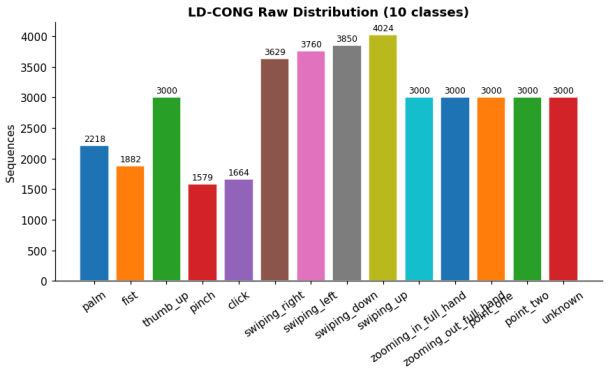
ax = axes[0]
bars = ax.bar(
    class_labels_ld, counts_all,
    color=sns.color_palette("tab10", len(class_labels_ld)), edgecolor="white
)
ax.bar_label(bars, fmt="%d", fontsize=8, padding=2)
ax.set_title("LD-CONG Raw Distribution (10 classes)", fontweight="bold")
ax.set_ylabel("Sequences")
ax.tick_params(axis="x", rotation=35)

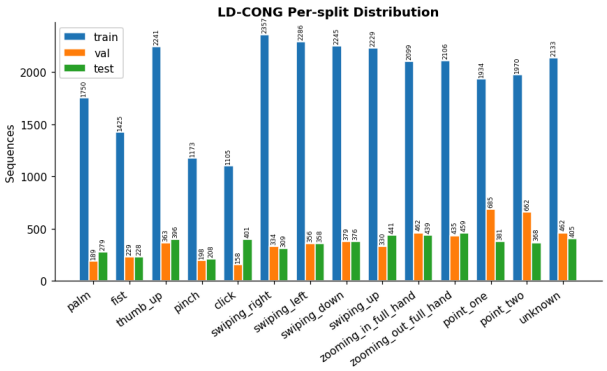
ax = axes[1]
b1 = ax.bar(x - w, counts_train, width=w, label="train", edgecolor="white")
b2 = ax.bar(x,     counts_val,   width=w, label="val",   edgecolor="white")
b3 = ax.bar(x + w, counts_test,  width=w, label="test",  edgecolor="white")
```

```
ax.bar_label(b1, fmt="%d", fontsize=6, padding=2, rotation=90)
ax.bar_label(b2, fmt="%d", fontsize=6, padding=2, rotation=90)
ax.bar_label(b3, fmt="%d", fontsize=6, padding=2, rotation=90)
ax.set_title("LD-CONG Per-split Distribution", fontweight="bold")
ax.set_ylabel("Sequences")
ax.set_xticks(x)
ax.set_xticklabels(class_labels_ld, rotation=35, ha="right")
ax.legend()

plt.tight_layout()
plt.show()

# Imbalance check
min_c, max_c = min(counts_all), max(counts_all)
print(f"\nMin class count : {min_c}  |  Max class count : {max_c}  |  Ratio
if max_c / max(min_c, 1) > 5:
    print("⚠  High imbalance — consider increasing max_samples_per_class or
else:
    print("✓ Class balance within acceptable range.")
```






```
Min class count : 1579  |  Max class count : 4024  |  Ratio : 2.5×
✓ Class balance within acceptable range.
```

In [ ]:

```
# ── Section 3b — Class distribution table ──────────────────────────
import pandas as pd

_rows = []
for cls in class_labels_ld:
    _rows.append({
        "Class":        cls,
        "Train":        dist_train.get(cls, 0),
        "Val":          dist_val.get(cls, 0),
        "Test":         dist_test.get(cls, 0),
        "Total":        dist_all.get(cls, 0),
        "Train %":      f"{dist_train.get(cls, 0) / max(dist_all.get(cls,1),
        "Val %":        f"{dist_val.get(cls, 0)   / max(dist_all.get(cls,1),
        "Test %":       f"{dist_test.get(cls, 0)  / max(dist_all.get(cls,1),
    })

_df = pd.DataFrame(_rows).set_index("Class")

# Totals row
_totals = pd.DataFrame([{
    "Class": "TOTAL",
    "Train": _df["Train"].sum(),
```

```
    "Val":   _df["Val"].sum(),
    "Test":  _df["Test"].sum(),
    "Total": _df["Total"].sum(),
    "Train %": f"{_df['Train'].sum() / max(_df['Total'].sum(), 1):.0%}",
    "Val %":   f"{_df['Val'].sum()   / max(_df['Total'].sum(), 1):.0%}",
    "Test %":  f"{_df['Test'].sum()  / max(_df['Total'].sum(), 1):.0%}",
}]).set_index("Class")

_display_df = pd.concat([_df, _totals])

def _highlight_total(row):
    return ["font-weight: bold; background-color: #f0f0f0"] * len(row) \
        if row.name == "TOTAL" else [""] * len(row)

display(
    _display_df.style
    .apply(_highlight_total, axis=1)
    .set_caption("LD-CONG Hybrid — Dataset split distribution")
    .set_table_styles([
        {"selector": "caption", "props": [("font-size", "13px"), ("font-weig
        {"selector": "th",      "props": [("text-align", "center")]},
        {"selector": "td",      "props": [("text-align", "right"), ("padding
    ])
    .bar(subset=["Train", "Val", "Test"], color="#b3d1ff", vmin=0)
)
```

4. Model Architecture Preview

In [33]:

```
from maestro.training.train_lstm import _build_model

model_preview = _build_model(jester_config.model, DEVICE)
n_params     = sum(p.numel() for p in model_preview.parameters())
n_trainable  = sum(p.numel() for p in model_preview.parameters() if p.requir

print(model_preview)
print()
print(f"Total params     : {n_params:,}")
print(f"Trainable params : {n_trainable:,}")
del model_preview
```

```
EnhancedTwoStreamLSTM(
  (proj_a): Sequential(
    (0): Linear(in_features=73, out_features=96, bias=True)
    (1): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
    (2): GELU(approximate='none')
  )
  (proj_b): Sequential(
    (0): Linear(in_features=74, out_features=96, bias=True)
    (1): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
    (2): GELU(approximate='none')
  )
  (lstm_a): LSTM(96, 96, num_layers=4, batch_first=True, dropout=0.35, bidir
ectional=True)
  (lstm_b): LSTM(96, 96, num_layers=4, batch_first=True, dropout=0.35, bidir
ectional=True)
  (ln_a): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (ln_b): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (mha_a): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=192, out_feature
s=192, bias=True)
  )
  (mha_b): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=192, out_feature
s=192, bias=True)
  )
  (ln_post_a): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (ln_post_b): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (pool_ln_a): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (pool_ln_b): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (gate_a): Sequential(
    (0): Linear(in_features=192, out_features=96, bias=True)
    (1): Tanh()
    (2): Linear(in_features=96, out_features=192, bias=True)
    (3): Sigmoid()
  )
  (gate_b): Sequential(
    (0): Linear(in_features=192, out_features=96, bias=True)
    (1): Tanh()
    (2): Linear(in_features=96, out_features=192, bias=True)
    (3): Sigmoid()
  )
  (ln_cross_a): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (ln_cross_b): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (head): Sequential(
    (0): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
    (1): Linear(in_features=384, out_features=192, bias=True)
    (2): GELU(approximate='none')
    (3): Dropout(p=0.35, inplace=False)
    (4): Linear(in_features=192, out_features=27, bias=True)
  )
)

Total params     : 2,102,715
Trainable params : 2,102,715
```

5. Phase 1 — Jester Pretraining (or load existing checkpoint)

If `PRETRAINED_CHECKPOINT` is set and exists, Phase 1 is **skipped** and the existing checkpoint is used directly. This is the recommended flow when you already have a stable Jester pretrained model from notebook 03 or 07.

In [34]:
```
from maestro.training.train_lstm import TrainingResult, train_lstm

if PRETRAINED_CHECKPOINT is not None and PRETRAINED_CHECKPOINT.exists():
    print(f"✓ Re-using existing Jester checkpoint: {PRETRAINED_CHECKPOINT.na
    print("  Phase 1 training skipped.")
    pretrain_result = None
else:
    print("Running Phase 1 Jester pretraining from scratch…")
    pretrain_result: TrainingResult = train_lstm(
        dataset_path=JESTER_NPZ,
        config=jester_config,
        output_dir=CHECKPOINT_DIR,
        run_root_dir=RUN_ROOT_DIR,
        device=DEVICE,
    )
    PRETRAINED_CHECKPOINT = pretrain_result.checkpoint_path
    print()
    print(f"Phase 1 checkpoint : {PRETRAINED_CHECKPOINT}")

print(f"\nPretrained checkpoint : {PRETRAINED_CHECKPOINT}")
```

```
Running Phase 1 Jester pretraining from scratch…
```

```
Training  two_stream_attn_v1_20260513T155730Z
  Model     : EnhancedTwoStreamLSTM  (2,102,715 params)
  Dataset   : train=49,205  val=5,796
  Optimiser : AdamW  lr=1.00e-04  wd=5.00e-04
  Schedule  : ReduceLROnPlateau  factor=0.5  patience=8
  Stopping  : patience=15  min_delta=1e-04  metric=val_macro_f1
--------------------------------------------------------------------------------
---------
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1        LR    Time
ES  Note
--------------------------------------------------------------------------------
---------
     1/80       3.3356     11.7%    2.3666    35.1%   45.2%  1.00e-04    7.0
s  * best
     2/80       2.2016     45.5%    1.8816    62.8%   68.4%  1.00e-04    6.7
s  * best
     3/80       1.9100     58.9%    1.7011    73.9%   76.2%  1.00e-04    6.7
s  * best
     4/80       1.7606     66.0%    1.6143    75.6%   79.0%  1.00e-04    6.8
s  * best
     5/80       1.6690     69.8%    1.5751    76.9%   79.4%  1.00e-04    6.8
s  * best
     6/80       1.6193     72.9%    1.5207    78.6%   81.4%  1.00e-04    6.8
s  * best
     7/80       1.5761     74.9%    1.4995    80.4%   82.5%  1.00e-04    6.8
s  * best
     8/80       1.5504     76.7%    1.4973    80.3%   82.9%  1.00e-04    6.9
s  * best
     9/80       1.5217     78.6%    1.4913    82.7%   83.6%  1.00e-04    6.8
s  * best
    10/80       1.4975     79.8%    1.4672    84.8%   85.1%  1.00e-04    6.9
s  * best
    11/80       1.4814     81.4%    1.4769    84.8%   85.1%  1.00e-04    6.8
s    ES 1/15
    12/80       1.4646     82.4%    1.4555    85.1%   85.4%  1.00e-04    6.8
s  * best
    13/80       1.4491     83.7%    1.4547    85.5%   85.6%  1.00e-04    6.8
s  * best
    14/80       1.4381     84.6%    1.4262    86.7%   86.4%  1.00e-04    6.8
s  * best
    15/80       1.4320     85.2%    1.4408    86.0%   85.5%  1.00e-04    6.9
s    ES 1/15
    16/80       1.4212     85.7%    1.4347    87.5%   86.9%  1.00e-04    6.8
s  * best
    17/80       1.4106     86.6%    1.4204    87.0%   86.6%  1.00e-04    6.8
s    ES 1/15
    18/80       1.3981     87.3%    1.4167    88.2%   87.1%  1.00e-04    6.8
s  * best
    19/80       1.3926     87.8%    1.4268    88.3%   87.1%  1.00e-04    6.8
s  * best
    20/80       1.3840     88.2%    1.4202    89.1%   87.7%  1.00e-04    6.8
s  * best
    21/80       1.3800     88.6%    1.4095    88.5%   87.3%  1.00e-04    6.8
s    ES 1/15
    22/80       1.3718     89.1%    1.4170    89.6%   88.0%  1.00e-04    6.9
s  * best
```

```
    23/80       1.3667     89.5%    1.3979    89.7%   87.9%  1.00e-04    6.9
s    ES 1/15
    24/80       1.3608     89.8%    1.4060    89.7%   88.3%  1.00e-04    6.8
s  * best
    25/80       1.3512     90.2%    1.4153    89.8%   88.0%  1.00e-04    6.8
s    ES 1/15
    26/80       1.3466     90.5%    1.4022    89.4%   87.8%  1.00e-04    6.8
s    ES 2/15
    27/80       1.3428     90.7%    1.4107    89.1%   87.4%  1.00e-04    6.8
s    ES 3/15
    28/80       1.3388     91.2%    1.3983    90.2%   88.7%  1.00e-04    6.8
s  * best
    29/80       1.3303     91.4%    1.4025    90.4%   88.6%  1.00e-04    6.9
s    ES 1/15
    30/80       1.3323     91.5%    1.3905    90.6%   88.6%  1.00e-04    6.8
s    ES 2/15
    31/80       1.3252     91.9%    1.4034    89.7%   88.1%  1.00e-04    6.8
s    ES 3/15
    32/80       1.3214     91.9%    1.4033    90.3%   88.5%  1.00e-04    6.8
s    ES 4/15
    33/80       1.3154     92.4%    1.3990    90.9%   89.0%  1.00e-04    6.8
s  * best
    34/80       1.3108     92.6%    1.3938    90.6%   88.9%  1.00e-04    6.8
s    ES 1/15
    35/80       1.3082     92.7%    1.4127    89.9%   88.0%  1.00e-04    6.7
s    ES 2/15
    36/80       1.3075     92.7%    1.3979    91.0%   89.1%  1.00e-04    6.8
s  * best
    37/80       1.3022     93.0%    1.3971    90.3%   88.4%  1.00e-04    6.7
s    ES 1/15
    38/80       1.2998     93.2%    1.3931    90.8%   89.0%  1.00e-04    6.7
s    ES 2/15
    39/80       1.2938     93.4%    1.3974    91.0%   89.2%  1.00e-04    6.7
s  * best
    40/80       1.2904     93.5%    1.3996    91.1%   89.3%  1.00e-04    6.7
s  * best
    41/80       1.2937     93.6%    1.3982    90.8%   88.7%  1.00e-04    6.8
s    ES 1/15
    42/80       1.2874     93.7%    1.3838    90.6%   88.8%  1.00e-04    6.8
s    ES 2/15
    43/80       1.2817     94.1%    1.4101    91.2%   88.9%  1.00e-04    6.8
s    ES 3/15
    44/80       1.2843     94.0%    1.3916    91.1%   89.0%  1.00e-04    6.9
s    ES 4/15
    45/80       1.2779     94.2%    1.3991    91.1%   88.8%  1.00e-04    6.8
s    ES 5/15
    46/80       1.2759     94.4%    1.3947    91.3%   89.2%  1.00e-04    6.8
s    ES 6/15
    47/80       1.2752     94.5%    1.3963    91.0%   88.9%  1.00e-04    6.8
s    ES 7/15
    48/80       1.2708     94.6%    1.3956    91.3%   89.1%  1.00e-04    6.8
s    ES 8/15
    49/80       1.2670     94.7%    1.4031    91.4%   89.1%  5.00e-05    6.8
s    ES 9/15
    50/80       1.2593     95.3%    1.3902    91.3%   89.2%  5.00e-05    6.8
s    ES 10/15
```

```
    51/80       1.2548     95.3%    1.3910    91.3%   89.1%  5.00e-05    6.8
s    ES 11/15
    52/80       1.2552     95.4%    1.3936    91.3%   89.2%  5.00e-05    6.7
s    ES 12/15
    53/80       1.2525     95.5%    1.3959    91.4%   89.2%  5.00e-05    6.7
s    ES 13/15
    54/80       1.2512     95.6%    1.3995    91.5%   89.1%  5.00e-05    6.7
s    ES 14/15
    55/80       1.2514     95.6%    1.3923    91.5%   89.3%  5.00e-05    6.7
s    ES 15/15

Early stopping at epoch 55  (no improvement in val_macro_f1 for 15 epochs)

Best epoch  : 40  val_macro_f1=0.8925  val_loss=1.3996

Phase 1 checkpoint : /home/divclab/.hslay13/Maestro/models/checkpoints/two_s
tream_attn_v1_20260513T155730Z.pt

Pretrained checkpoint : /home/divclab/.hslay13/Maestro/models/checkpoints/tw
o_stream_attn_v1_20260513T155730Z.pt
```

6. Phase 1 Learning Curves (optional — if trained above)

In [35]:

```python
if pretrain_result is None:
    print("Phase 1 was skipped (re-using checkpoint) — no learning curves to
else:
    history_p1 = pretrain_result.train_history
    epochs_p1  = [h["epoch"] for h in history_p1]

    fig, axes = plt.subplots(1, 3, figsize=(16, 4))

    ax = axes[0]
    ax.plot(epochs_p1, [h["train_loss"] for h in history_p1], label="train",
    ax.plot(epochs_p1, [h["val_loss"] for h in history_p1], label="val", lir
    ax.set_title("Loss", fontweight="bold")
    ax.set_xlabel("Epoch")
    ax.legend(fontsize=9)

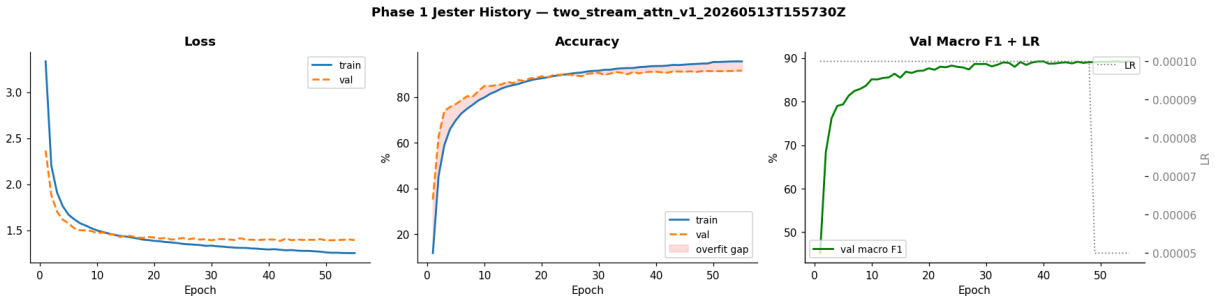
    ax = axes[1]
    t = [h["train_accuracy"] * 100 for h in history_p1]
    v = [h["val_accuracy"] * 100 for h in history_p1]
    ax.plot(epochs_p1, t, label="train", linewidth=1.8)
    ax.plot(epochs_p1, v, label="val",   linewidth=1.8, linestyle="--")
    ax.fill_between(epochs_p1, v, t, alpha=0.12, color="red", label="overfit
    ax.set_title("Accuracy", fontweight="bold")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("%")
    ax.legend(fontsize=9)

    ax = axes[2]
    ax.plot(
        epochs_p1,
        [h.get("val_macro_f1", float("nan")) * 100 for h in history_p1],
```

```
        label="val macro F1", linewidth=1.8, color="green",
    )
    ax2 = ax.twinx()
    ax2.plot(
        epochs_p1,
        [h.get("learning_rate", float("nan")) for h in history_p1],
        label="LR", linewidth=1.2, color="grey", linestyle=":",
    )
    ax2.set_ylabel("LR", color="grey")
    ax2.tick_params(axis="y", labelcolor="grey")
    ax.set_title("Val Macro F1 + LR", fontweight="bold")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("%")
    ax.legend(fontsize=9, loc="lower left")
    ax2.legend(fontsize=9, loc="upper right")

    fig.suptitle(
        f"Phase 1 Jester History — {pretrain_result.model_version}",
        fontsize=12, fontweight="bold",
    )
    plt.tight_layout()
    plt.show()

    best_p1 = max(history_p1, key=lambda h: h.get("val_macro_f1", 0.0))
    print(f"Best epoch : {int(best_p1['epoch'])}   Val macro F1 : {best_p1.g
```




```
Best epoch : 55   Val macro F1 : 89.3%
```

7. Phase 2 — LD-CONG Fine-tuning (Stage A + B)

- **Stage A** (`FREEZE_EPOCHS` epochs): only the new 10-class head is trained; backbone frozen.
- **Stage B** (remaining epochs): full model with EWC penalty (`EWC_WEIGHT=2000.0`).

Backbone warmup at Stage B onset

When transitioning from Stage A to Stage B, the backbone is unfrozen with a **linear LR warmup** over `STAGE_B_WARMUP_EPOCHS` epochs. This prevents the loss spike caused by the cold AdamW second-moment state of previously-frozen backbone parameters.

EWC-only strategy

Based on catastrophic-forgetting experiments on this codebase, **EWC alone** (`EWC_WEIGHT=2000`) is the production anti-forgetting method:

- EWC=2000 → **−0.9 pp** Jester drop after fine-tuning ✓
- LLRD → −13.1 pp ✗
- Progressive unfreezing → −3.4 to −5.6 pp ✗ Distillation, replay-CE, and GPM are therefore disabled.

In [36]:
```
from maestro.training.train_lstm import finetune_lstm

finetune_result: TrainingResult = finetune_lstm(
    dataset_path=LD_CONG_NPZ,
    config=ld_cong_config,
    pretrained_checkpoint_path=PRETRAINED_CHECKPOINT,
    output_dir=CHECKPOINT_DIR,
    run_root_dir=RUN_ROOT_DIR,
    device=DEVICE,
    freeze_epochs=FREEZE_EPOCHS,
    stage_b_warmup_epochs=STAGE_B_WARMUP_EPOCHS,
    backbone_lr_multiplier=BACKBONE_LR_MULTIPLIER,
    ewc_weight=EWC_WEIGHT,
    forgetting_penalty_weight=FORGETTING_PENALTY_WEIGHT,
    # Disabled (EWC-only production strategy):
    replay_dataset_path=None,
    replay_max_samples_per_class=REPLAY_MAX_SAMPLES_PER_CLASS,
    distillation_weight=DISTILLATION_WEIGHT,
    replay_ce_weight=REPLAY_CE_WEIGHT,
    gpm_components=GPM_COMPONENTS,
)

print()
print(f"Fine-tuned checkpoint : {finetune_result.checkpoint_path}")
print(f"Run dir               : {finetune_result.run_dir}")
```

```
Fine-tuning  two_stream_attn_v1_finetune_20260513T160347Z
  Pretrained : two_stream_attn_v1_20260513T155730Z.pt
  Model      : EnhancedTwoStreamLSTM  (2,100,206 params)
  Dataset    : train=27,053  val=5,242
  Classes    : 14  ['palm', 'fist', 'thumb_up', 'pinch', 'click', 'swiping_r
ight', 'swiping_left', 'swiping_down', 'swiping_up', 'zooming_in_full_hand',
'zooming_out_full_hand', 'point_one', 'point_two', 'unknown']

──────────────────────────────────────────────────────────────────────────────
────
Stage A — backbone frozen, head-only  (10 epochs, lr=1e-3)
------------------------------------------------------------------------------
----
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1
------------------------------------------------------------------------------
----
```

```
     1/10       0.6911     87.7%    0.5508    92.1%   92.3%
     2/10       0.5430     92.6%    0.5246    93.3%   93.6%
     3/10       0.5174     93.7%    0.5137    93.3%   93.5%
     4/10       0.5024     94.2%    0.5084    93.5%   93.7%
     5/10       0.4937     94.6%    0.5005    94.3%   94.9%
     6/10       0.4855     94.9%    0.5033    94.3%   94.8%
     7/10       0.4792     95.2%    0.5052    94.1%   94.6%
     8/10       0.4743     95.3%    0.4987    94.3%   95.0%
     9/10       0.4697     95.5%    0.4981    94.5%   95.2%
    10/10       0.4682     95.5%    0.4945    94.7%   95.2%

Stage A complete. Val macro-F1 = 0.9525

──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────
Stage B — full fine-tuning  (up to 60 epochs, head_lr=3.00e-05, backbone_lr=
3.00e-06  [warmup 5 epochs: 0→target])
  EWC regularisation  ewc_weight=2000.0
------------------------------------------------------------------------------
---------------------------------
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1   JstrF1        LR
Distill  ReplayCE       EWC  Note
------------------------------------------------------------------------------
---------------------------------
     1/60       0.4536     96.1%    0.4893    94.9%   95.5%      nan  3.00e-
05       nan       nan       nan  * best
     2/60       0.4525     96.1%    0.4865    95.1%   95.7%      nan  3.00e-
05       nan       nan       nan  * best
     3/60       0.4490     96.3%    0.4848    95.2%   95.7%      nan  3.00e-
05       nan       nan       nan  * best
     4/60       0.4456     96.4%    0.4830    95.2%   95.7%      nan  3.00e-
05       nan       nan       nan    ES 1/12
     5/60       0.4435     96.5%    0.4826    95.2%   95.7%      nan  3.00e-
05       nan       nan       nan    ES 2/12
     6/60       0.4395     96.6%    0.4792    95.3%   95.8%      nan  3.00e-
05       nan       nan       nan  * best
     7/60       0.4359     96.7%    0.4766    95.4%   96.0%      nan  3.00e-
05       nan       nan       nan  * best
     8/60       0.4353     96.7%    0.4752    95.4%   96.0%      nan  3.00e-
05       nan       nan       nan    ES 1/12
     9/60       0.4326     96.8%    0.4738    95.4%   96.0%      nan  3.00e-
05       nan       nan       nan    ES 2/12
    10/60       0.4303     97.0%    0.4731    95.4%   95.9%      nan  3.00e-
05       nan       nan       nan    ES 3/12
    11/60       0.4300     96.9%    0.4707    95.5%   96.0%      nan  3.00e-
05       nan       nan       nan  * best
    12/60       0.4260     97.1%    0.4718    95.4%   95.9%      nan  3.00e-
05       nan       nan       nan    ES 1/12
    13/60       0.4259     97.0%    0.4693    95.5%   96.1%      nan  3.00e-
05       nan       nan       nan  * best
    14/60       0.4247     97.1%    0.4684    95.4%   95.9%      nan  3.00e-
05       nan       nan       nan    ES 1/12
    15/60       0.4225     97.2%    0.4685    95.5%   96.0%      nan  3.00e-
05       nan       nan       nan    ES 2/12
    16/60       0.4233     97.1%    0.4672    95.6%   96.2%      nan  3.00e-
05       nan       nan       nan  * best
```

```
    17/60       0.4200     97.3%    0.4652    95.7%   96.1%      nan  3.00e-
05       nan       nan       nan    ES 1/12
    18/60       0.4213     97.2%    0.4651    95.7%   96.1%      nan  3.00e-
05       nan       nan       nan    ES 2/12
    19/60       0.4184     97.2%    0.4648    95.6%   96.1%      nan  3.00e-
05       nan       nan       nan    ES 3/12
    20/60       0.4170     97.3%    0.4636    95.6%   96.2%      nan  3.00e-
05       nan       nan       nan    ES 4/12
    21/60       0.4154     97.4%    0.4626    95.6%   96.1%      nan  3.00e-
05       nan       nan       nan    ES 5/12
    22/60       0.4153     97.5%    0.4632    95.6%   96.1%      nan  3.00e-
05       nan       nan       nan    ES 6/12
    23/60       0.4140     97.5%    0.4613    95.7%   96.2%      nan  3.00e-
05       nan       nan       nan    ES 7/12
    24/60       0.4115     97.6%    0.4630    95.5%   96.0%      nan  3.00e-
05       nan       nan       nan    ES 8/12
    25/60       0.4111     97.6%    0.4608    95.7%   96.2%      nan  1.50e-
05       nan       nan       nan    ES 9/12
    26/60       0.4100     97.6%    0.4605    95.7%   96.1%      nan  1.50e-
05       nan       nan       nan    ES 10/12
    27/60       0.4093     97.6%    0.4606    95.7%   96.2%      nan  1.50e-
05       nan       nan       nan    ES 11/12
    28/60       0.4112     97.5%    0.4607    95.7%   96.2%      nan  1.50e-
05       nan       nan       nan    ES 12/12

Early stopping at epoch 28  (no improvement for 12 epochs)

Best epoch : 16  val_macro_f1=0.9616  composite=0.9616  val_loss=0.4672

Fine-tuned checkpoint : /home/divclab/.hslay13/Maestro/models/checkpoints/tw
o_stream_attn_v1_finetune_20260513T160347Z.pt
Run dir               : /home/divclab/.hslay13/Maestro/experiments/runs/two_
stream_attn_v1_finetune_20260513T160347Z
```

8. Stage Summary

In [37]:

```python
history_ft   = finetune_result.train_history
stage_a      = [h for h in history_ft if h.get("stage", 1) == 0]
stage_b      = [h for h in history_ft if h.get("stage", 1) == 1]
stage_b_warmup = [h for h in stage_b if h.get("stage_b_warmup", False)]

print("=" * 70)
print("FINE-TUNING STAGES")
print("=" * 70)
print(f"  Stage A (frozen backbone)     : {len(stage_a)} epochs")
print(f"  Stage B backbone warmup       : {len(stage_b_warmup)} epochs  (LR
print(f"  Stage B full fine-tuning      : {len(stage_b) - len(stage_b_warmup
if stage_b:
    best_b        = max(stage_b, key=lambda h: h.get("val_macro_f1", 0))
    best_comp     = max(stage_b, key=lambda h: h.get("composite_score", 0))
    best_b_f1     = best_b.get("val_macro_f1", 0)
    best_b_epoch  = int(best_b["epoch"])
    best_comp_score = best_comp.get("composite_score", 0)
    best_comp_epoch = int(best_comp["epoch"])
```

```
    print(f"  Best Stage B val_macro_f1     : {best_b_f1:.1%}    epoch : {be
    print(f"  Best composite score          : {best_comp_score:.4f}    epoch
```

```
======================================================================
FINE-TUNING STAGES
======================================================================
  Stage A (frozen backbone)     : 10 epochs
  Stage B backbone warmup       : 5 epochs  (LR 0 → target)
  Stage B full fine-tuning      : 23 epochs
  Best Stage B val_macro_f1     : 96.2%    epoch : 26
  Best composite score          : 0.9616    epoch : 26
```

9. Learning Curves (Phase 2)

In [38]:

```
history = finetune_result.train_history
epochs  = [h["epoch"] for h in history]
stage_a_count  = len([h for h in history if h.get("stage", 1) == 0])
boundary_epoch = epochs[stage_a_count - 1] + 0.5 if stage_a_count > 0 else N

stage_b_history = [h for h in history if h.get("stage", 1) == 1]
warmup_epochs_b = [h["epoch"] for h in stage_b_history if h.get("stage_b_war
warmup_end_epoch = (warmup_epochs_b[-1] + 0.5) if warmup_epochs_b else None


def _shade_warmup(ax_):
    if boundary_epoch and warmup_end_epoch:
        ax_.axvspan(boundary_epoch, warmup_end_epoch, alpha=0.08, color="ora


fig, axes = plt.subplots(1, 3, figsize=(20, 5))

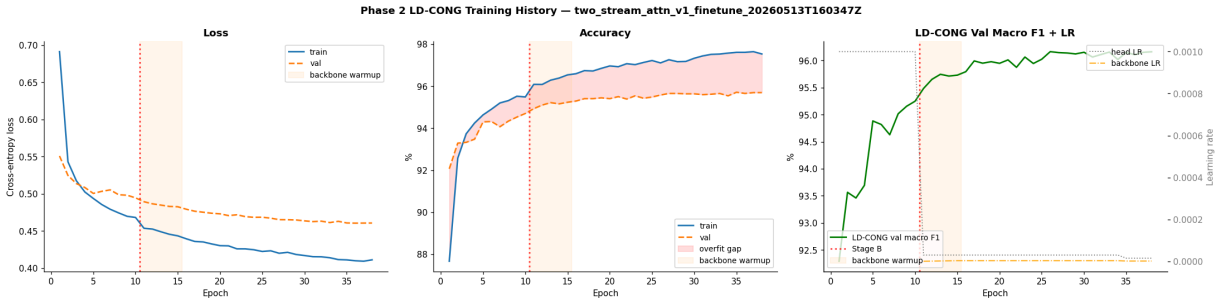
# Panel 1: Loss
ax = axes[0]
ax.plot(epochs, [h["train_loss"] for h in history], label="train", linewidth
ax.plot(epochs, [h["val_loss"]   for h in history], label="val",   linewidth
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=2, alph
_shade_warmup(ax)
ax.set_title("Loss", fontweight="bold")
ax.set_xlabel("Epoch")
ax.set_ylabel("Cross-entropy loss")
ax.legend(fontsize=9)

# Panel 2: Accuracy
ax = axes[1]
train_accs = [h["train_accuracy"] * 100 for h in history]
val_accs   = [h["val_accuracy"]   * 100 for h in history]
ax.plot(epochs, train_accs, label="train", linewidth=1.8)
ax.plot(epochs, val_accs,   label="val",   linewidth=1.8, linestyle="--")
ax.fill_between(epochs, val_accs, train_accs, alpha=0.12, color="red", label
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=2, alph
_shade_warmup(ax)
ax.set_title("Accuracy", fontweight="bold")
ax.set_xlabel("Epoch")
```

```
ax.set_ylabel("%")
ax.legend(fontsize=9)

# Panel 3: Val Macro F1 + LR
ax = axes[2]
ax.plot(
    epochs,
    [h.get("val_macro_f1", float("nan")) * 100 for h in history],
    label="LD-CONG val macro F1", linewidth=1.8, color="green",
)
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=2, alph
_shade_warmup(ax)
ax2 = ax.twinx()
ax2.plot(
    epochs,
    [h.get("learning_rate", float("nan")) for h in history],
    label="head LR", linewidth=1.2, color="grey", linestyle=":",
)
ax2.plot(
    [h["epoch"] for h in stage_b_history],
    [h.get("backbone_lr", float("nan")) for h in stage_b_history],
    label="backbone LR", linewidth=1.2, color="orange", linestyle="-.",
)
ax2.set_ylabel("Learning rate", color="grey")
ax2.tick_params(axis="y", labelcolor="grey")
ax.set_title("LD-CONG Val Macro F1 + LR", fontweight="bold")
ax.set_xlabel("Epoch")
ax.set_ylabel("%")
ax.legend(fontsize=9, loc="lower left")
ax2.legend(fontsize=9, loc="upper right")

fig.suptitle(
    f"Phase 2 LD-CONG Training History — {finetune_result.model_version}",
    fontsize=12, fontweight="bold",
)
plt.tight_layout()
plt.show()
```



In [ ]:

```
# ── Section 9b — Overfitting gap ──────────────────────────────────────
_gap      = [h["train_accuracy"] * 100 - h["val_accuracy"] * 100 for h in hi
_epochs   = [h["epoch"] for h in history]

fig, ax = plt.subplots(figsize=(12, 4))

# Shaded regions
```

```
ax.axhspan(0,  5, alpha=0.06, color="green",  label="healthy (< 5 pp)")
ax.axhspan(5, 15, alpha=0.06, color="orange", label="mild overfit (5–15 pp)"
ax.axhspan(15, max(max(_gap) + 2, 16), alpha=0.06, color="red", label="overf

ax.plot(_epochs, _gap, color="#d62728", linewidth=2, label="train − val acc"
ax.fill_between(_epochs, 0, _gap,
                where=[g > 0 for g in _gap],
                color="#d62728", alpha=0.15)
ax.fill_between(_epochs, 0, _gap,
                where=[g < 0 for g in _gap],
                color="#2ca02c", alpha=0.15)

# Stage boundary
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=2,
               alpha=0.7, label="Stage A→B")
if warmup_end_epoch:
    ax.axvspan(boundary_epoch, warmup_end_epoch, alpha=0.10,
               color="orange", label="backbone warmup")

ax.axhline(0, color="black", linewidth=0.8, linestyle="-")
ax.axhline(5, color="orange", linewidth=0.8, linestyle="--", alpha=0.6)
ax.axhline(15, color="red", linewidth=0.8, linestyle="--", alpha=0.6)

ax.set_title(
    f"Overfitting Gap — Phase 2 LD-CONG Fine-tuning\n"
    f"Final gap: {_gap[-1]:+.1f} pp  |  Max gap: {max(_gap):+.1f} pp  (epoch
    fontweight="bold",
)
ax.set_xlabel("Epoch")
ax.set_ylabel("Train acc − Val acc  (pp)")
ax.legend(fontsize=9, loc="upper right")
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)
plt.tight_layout()
plt.show()
```

10. Test Set Evaluation

In [39]:

```
ld_cong_test = finetune_result.test_report

print("=" * 70)
print("PHASE 2 TEST RESULTS (LD-CONG 10-class)")
print("=" * 70)
print(f"  Accuracy     : {ld_cong_test.accuracy:.1%}")
print(f"  Macro F1     : {ld_cong_test.macro_f1:.1%}")
print(f"  Loss         : {ld_cong_test.loss:.4f}")
print(f"  Calib. ECE   : {ld_cong_test.calibration_ece:.4f}")

# Jester retention
if stage_b_history:
    jester_retained = stage_b_history[-1].get("jester_val_f1", float("nan"))
    if jester_retained == jester_retained:
        print()
```

```
        print(f"  Jester retention F1 : {jester_retained:.1%}  (post-fine-tu

# Convenience aliases for downstream cells.
test_report = ld_cong_test
config      = ld_cong_config
result      = finetune_result
```

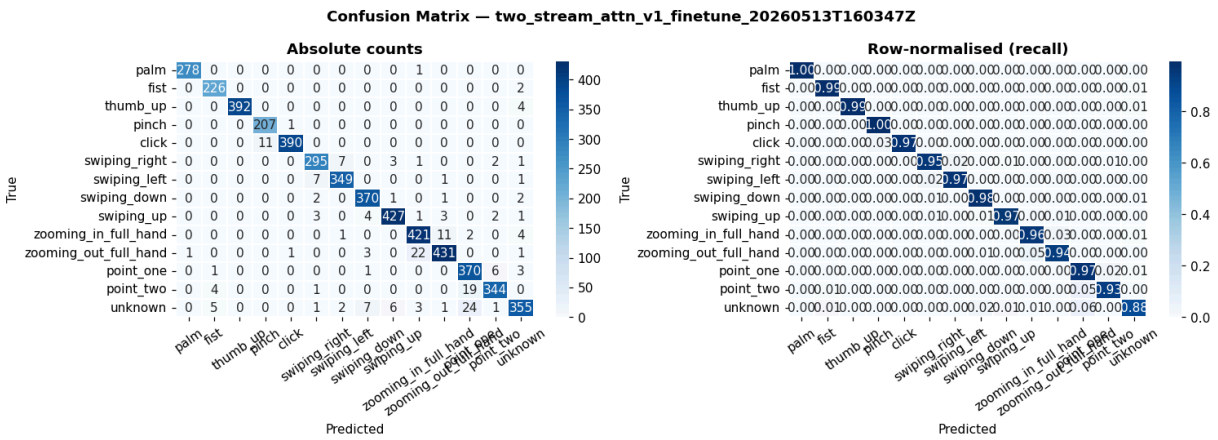
```
======================================================================
PHASE 2 TEST RESULTS (LD-CONG 10-class)
======================================================================
  Accuracy     : 96.2%
  Macro F1     : 96.3%
  Loss         : 0.4308
  Calib. ECE   : 0.0328
```

11. Confusion Matrix

In [40]:
```
class_labels_local = result.class_labels
cm      = np.array(test_report.confusion_matrix, dtype=np.float32)
cm_norm = cm / (cm.sum(axis=1, keepdims=True) + 1e-9)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, matrix, title, fmt in [
    (axes[0], cm,      "Absolute counts",          ".0f"),
    (axes[1], cm_norm, "Row-normalised (recall)",  ".2f"),
]:
    sns.heatmap(
        matrix,
        annot=True, fmt=fmt, cmap="Blues",
        xticklabels=class_labels_local, yticklabels=class_labels_local,
        linewidths=0.3, ax=ax, cbar=True,
    )
    ax.set_title(title, fontweight="bold")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    ax.tick_params(axis="x", rotation=35)
    ax.tick_params(axis="y", rotation=0)

fig.suptitle(f"Confusion Matrix — {result.model_version}", fontsize=12, font
plt.tight_layout()
plt.show()
```



12. Per-Class Recall

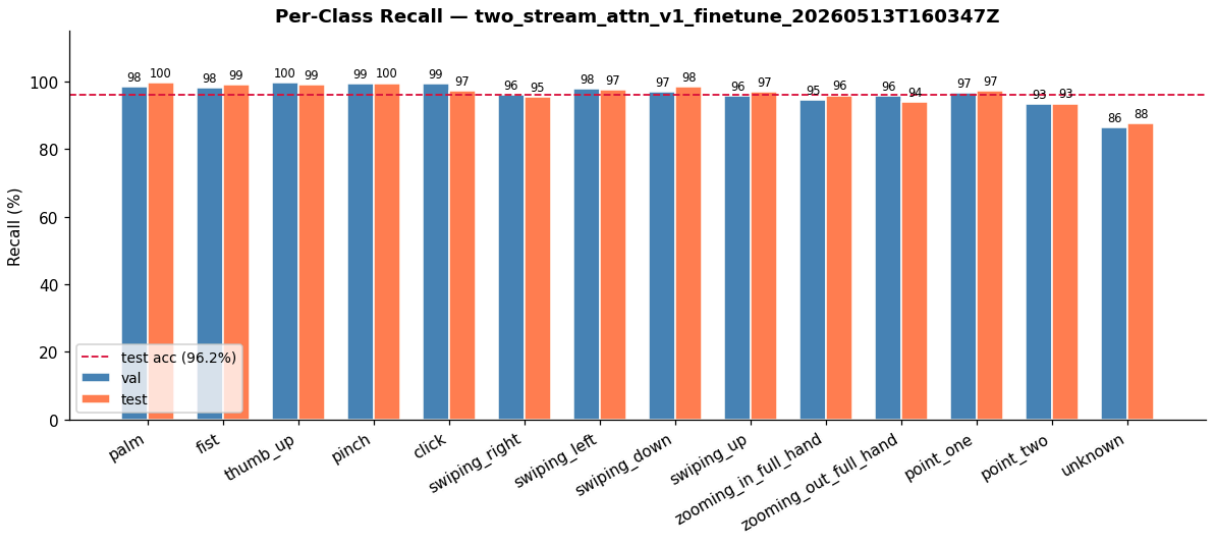
In [41]:

```
val_recall  = result.val_report.per_class_recall
test_recall = result.test_report.per_class_recall
classes     = list(test_recall.keys())

x     = np.arange(len(classes))
width = 0.35

fig, ax = plt.subplots(figsize=(11, 5))
bars1 = ax.bar(
    x - width / 2,
    [val_recall.get(c, 0) * 100 for c in classes],
    width, label="val", color="steelblue", edgecolor="white",
)
bars2 = ax.bar(
    x + width / 2,
    [test_recall.get(c, 0) * 100 for c in classes],
    width, label="test", color="coral", edgecolor="white",
)
ax.bar_label(bars1, fmt="%.0f", fontsize=7.5, padding=2)
ax.bar_label(bars2, fmt="%.0f", fontsize=7.5, padding=2)
ax.set_xticks(x)
ax.set_xticklabels(classes, rotation=30, ha="right")
ax.set_title(f"Per-Class Recall — {result.model_version}", fontweight="bold"
ax.set_ylabel("Recall (%)")
ax.axhline(
    100 * test_report.accuracy,
    color="crimson", linestyle="--", linewidth=1.2,
    label=f"test acc ({test_report.accuracy:.1%})",
)
ax.set_ylim(0, 115)
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

WEAK_THRESHOLD = 70.0
weak = [
    (cls, test_recall.get(cls, 0) * 100)
    for cls in classes
    if test_recall.get(cls, 0) * 100 < WEAK_THRESHOLD
]
if weak:
    print(f"⚠  Classes below {WEAK_THRESHOLD}% recall:")
    for cls, rec in sorted(weak, key=lambda item: item[1]):
        print(f"   {cls:<30} {rec:.1f}%")
else:
    print(f"✓ All classes above {WEAK_THRESHOLD}% recall.")
```



✓ All classes above 70.0% recall.

13. Final Checkpoint Reference

In [42]:

```
print("=" * 70)
print("FINAL OUTPUTS")
print("=" * 70)
print(f"  LD-CONG checkpoint : {finetune_result.checkpoint_path}")
print(f"  Run directory      : {finetune_result.run_dir}")
print(f"  Model version      : {finetune_result.model_version}")
print()
print(f"  Test accuracy      : {ld_cong_test.accuracy:.1%}")
print(f"  Test macro F1      : {ld_cong_test.macro_f1:.1%}")
print(f"  Calibration ECE    : {ld_cong_test.calibration_ece:.4f}")
print()
if stage_b_history:
    jester_retained = stage_b_history[-1].get("jester_val_f1", float("nan"))
    if jester_retained == jester_retained:
        print(f"  Jester retention F1 : {jester_retained:.1%}  (post-fine-tu
print()
print("Next step: use this checkpoint in run_realtime.py")
print("  model_config = configs/model/two_stream_attn_ld_cong.yaml")
```

```
======================================================================
FINAL OUTPUTS
======================================================================
  LD-CONG checkpoint : /home/divclab/.hslay13/Maestro/models/checkpoints/two
_stream_attn_v1_finetune_20260513T160347Z.pt
  Run directory      : /home/divclab/.hslay13/Maestro/experiments/runs/two_s
tream_attn_v1_finetune_20260513T160347Z
  Model version      : two_stream_attn_v1_finetune_20260513T160347Z

  Test accuracy      : 96.2%
  Test macro F1      : 96.3%
  Calibration ECE    : 0.0328


Next step: use this checkpoint in run_realtime.py
  model_config = configs/model/two_stream_attn_ld_cong.yaml
```

14. Export Inference Artifact

Saves a `.pt` inference artifact containing model weights, architecture config, class labels, and feature schema metadata — ready for `run_realtime.py`.

In [43]:

```python
from maestro.infrastructure.model.checkpoint_loader import load_inference_ar
from maestro.infrastructure.model.two_stream_attn_lstm import EnhancedTwoStr
from maestro.training.export_checkpoint import export_inference_artifact

artifact_path = export_inference_artifact(
    result=result,
    config=config,
    export_dir=EXPORTED_DIR,
)
print(f"Inference artifact : {artifact_path}")

# Verify artifact loads and infers correctly
art = load_inference_artifact(artifact_path=artifact_path, device=DEVICE)
art.model.eval()

assert isinstance(art.model, EnhancedTwoStreamLSTM), (
    f"Expected EnhancedTwoStreamLSTM, got {type(art.model).__name__}"
)

window_len_verify = int(result.dataset_metadata.get("window_length", 16))
dummy_verify = torch.zeros(1, window_len_verify, config.model.input_size, de
with torch.no_grad():
    out_verify = art.model(dummy_verify)

print(f"Model class      : {type(art.model).__name__}  ✓")
print(f"Schema version   : {art.feature_schema_version}")
print(f"Window length    : {art.window_length} frames")
print(f"Class labels     : {art.class_labels}")
print(
    f"Forward pass     : input {list(dummy_verify.shape)} → output {list(out
)
```

```
Inference artifact : /home/divclab/.hslay13/Maestro/models/exported/two_stre
am_attn_v1_finetune_20260513T160347Z_inference.pt
Model class      : EnhancedTwoStreamLSTM  ✓
Schema version   : feature-schema-v1
Window length    : 16 frames
Class labels     : ['palm', 'fist', 'thumb_up', 'pinch', 'click', 'swiping_r
ight', 'swiping_left', 'swiping_down', 'swiping_up', 'zooming_in_full_hand',
'zooming_out_full_hand', 'point_one', 'point_two', 'unknown']
Forward pass     : input [1, 16, 147] → output [1, 14]  ✓
```

15. (Optional) Publish to Hugging Face Hub

Set `HF_TOKEN` in environment before running.

In [44]:
```python
HF_TOKEN    = os.environ.get("HF_TOKEN", "")
HF_REPO_ID  = "ntsrigaud/maestro-lstm-ld-cong"
HF_PRIVATE  = True

if not HF_TOKEN:
    print("HF_TOKEN is not set — skipping Hugging Face upload.")
    print("Set HF_TOKEN environment variable to enable publishing.")
else:
    try:
        from maestro.training.hf_publish import publish_to_huggingface

        url = publish_to_huggingface(
            result=result,
            config=config,
            test_report=result.test_report,
            inference_artifact_path=artifact_path,
            repo_id=HF_REPO_ID,
            token=HF_TOKEN or None,
            private=HF_PRIVATE,
        )
        print(f"Published: {url}")
    except ImportError:
        print("huggingface_hub not installed. Run: pip install huggingface_h
```

```
HF_TOKEN is not set — skipping Hugging Face upload.
Set HF_TOKEN environment variable to enable publishing.
```